

**Versi 2.0**

Januari 2026



## **BASIS DATA**

### **MODUL 5 - FUNGSI AGREGAT**

**Pengarang:**

Dr.Ir. Agus Eko Minarno, S.Kom., M.Kom., IPM

Naufal Arkaan

Ismail Dwi Muh. Anugerah

**LABORATORIUM INFORMATIKA**

**Universitas Muhammadiyah Malang**

## DAFTAR ISI

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>DAFTAR ISI.....</b>                             | <b>2</b>  |
| <b>PENDAHULUAN.....</b>                            | <b>4</b>  |
| TUJUAN.....                                        | 4         |
| TARGET MODUL.....                                  | 4         |
| PERSIAPAN.....                                     | 4         |
| KATA KUNCI.....                                    | 4         |
| <b>BAHAN.....</b>                                  | <b>5</b>  |
| 1. Konsep Dasar Fungsi Agregat.....                | 5         |
| Agregat.....                                       | 5         |
| 2. Fungsi Agregat                                  |           |
| Dasar.....                                         | 5         |
| COUNT().....                                       | 5         |
| SUM().....                                         | 6         |
| AVG().....                                         | 6         |
| MIN().....                                         | 7         |
| MAX().....                                         | 7         |
| 3. Klausa Pendukung untuk Fungsi                   |           |
| Agregat.....                                       | 8         |
| 3.1 Klausa                                         |           |
| WHERE.....                                         | 8         |
| 3.2 Klausa GROUP BY.....                           | 9         |
| 3.3 Klausul MEMILIKI.....                          | 10        |
| 4. Mengurutkan Data (Mengurutkan Hasil Kueri)..... | 11        |
| Pengurutan Data.....                               | 11        |
| (ASC).....                                         | 11        |
| (DESC).....                                        | 12        |
| Agregat.....                                       | 12        |
| 5. Menggabungkan Fungsi Agregat dalam Kueri        |           |
| SQL.....                                           | 13        |
| Agregat.....                                       | 13        |
| BY.....                                            | 13        |
| HAVING.....                                        | 14        |
| BY.....                                            | 15        |
| 6. Fungsi Tambahan                                 |           |
| (Pendahuluan).....                                 | 15        |
| LISTAGG().....                                     | 16        |
| MEDIAN().....                                      | 16        |
| RANK().....                                        | 17        |
| dalam Fungsi Agregat.....                          | 17        |
| <b>BAHAN.....</b>                                  | <b>19</b> |
| <b>KEGIATAN LATIHAN.....</b>                       | <b>21</b> |
| <b>SUMBER BELAJAR DAN TUTORIAL.....</b>            | <b>23</b> |
| <b>DOCK.....</b>                                   | <b>24</b> |
| <b>TUGAS.....</b>                                  | <b>26</b> |



|                              |                     |
|------------------------------|---------------------|
| TUGAS 1.....                 | 26                  |
| TUGAS 2 (DEMO LANGSUNG)..... | 27 RUBRIK PENILAIAN |
| <b>PRAKTIKUM.....</b>        | <b>28</b>           |



## 4

## BAHAN

### 1. Konsep Dasar Fungsi Agregat

#### 1.1 Definisi Fungsi Agregat

Fungsi agregat adalah fungsi SQL yang digunakan untuk melakukan perhitungan pada sekumpulan baris dan mengembalikan **satu nilai ringkasan**. Fungsi-fungsi ini sering digunakan dalam analisis data untuk menghitung jumlah catatan, menghitung total, menghitung rata-rata, dan menemukan nilai minimum atau maksimum.

Fungsi agregat **mengabaikan nilai NULL**, kecuali untuk COUNT(\*), yang menghitung semua baris.

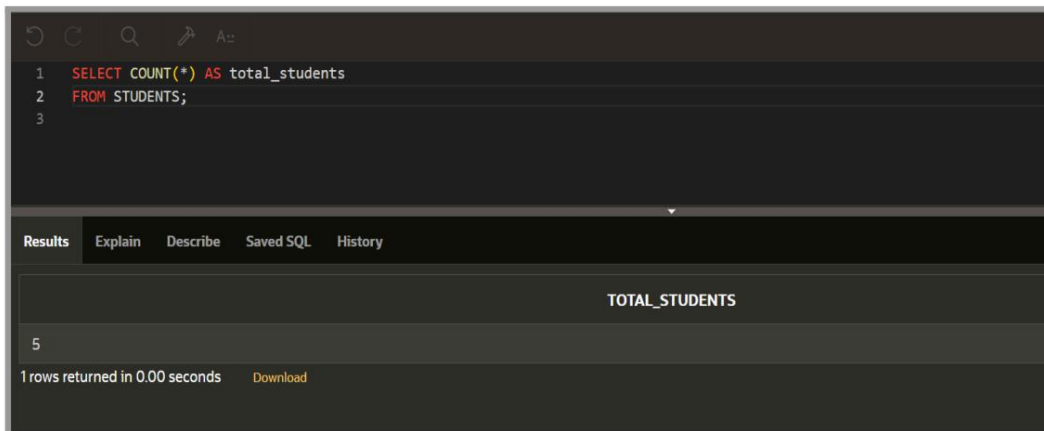
##### Analogi:

Alih-alih memeriksa setiap siswa secara individual, seorang dosen merangkum kinerja kelas menggunakan jumlah total siswa, IPK rata-rata, atau nilai tertinggi skor.

### 2. Fungsi Agregat Dasar

#### 2.1 Fungsi COUNT()

Fungsi COUNT() mengembalikan jumlah baris dalam sebuah tabel.



```
1 SELECT COUNT(*) AS total_students
2 FROM STUDENTS;
3
```

| TOTAL_STUDENTS |
|----------------|
| 5              |

1 rows returned in 0.00 seconds [Download](#)

##### Penjelasan:

COUNT(\*) menghitung semua baris dalam tabel STUDENTS.

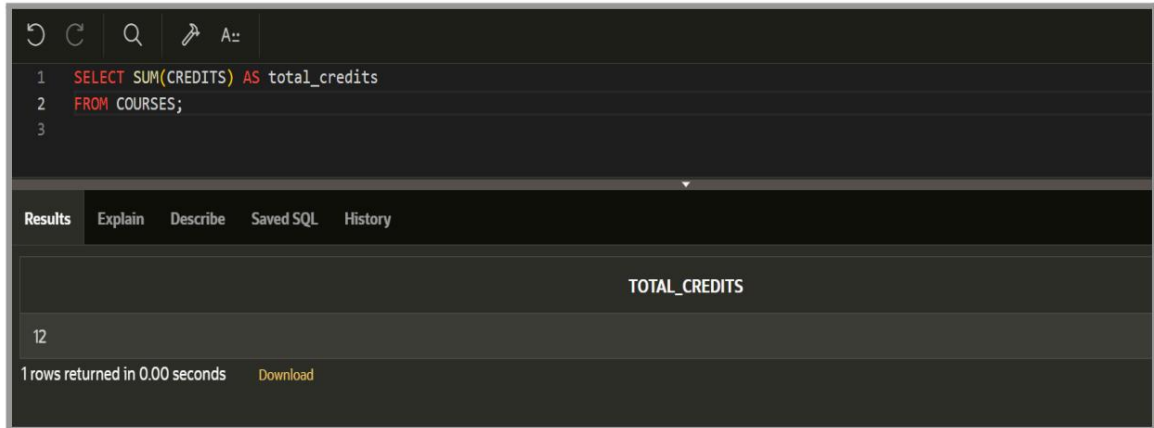


**Analogi:**

Menghitung jumlah mahasiswa yang terdaftar di universitas.

**2.2 Fungsi SUM()**

Fungsi SUM() menjumlahkan semua nilai numerik dalam sebuah kolom.



```

1 SELECT SUM(CREDITS) AS total_credits
2 FROM COURSES;
3

```

| TOTAL_CREDITS |
|---------------|
| 12            |

1 rows returned in 0.00 seconds [Download](#)

**Penjelasan:**

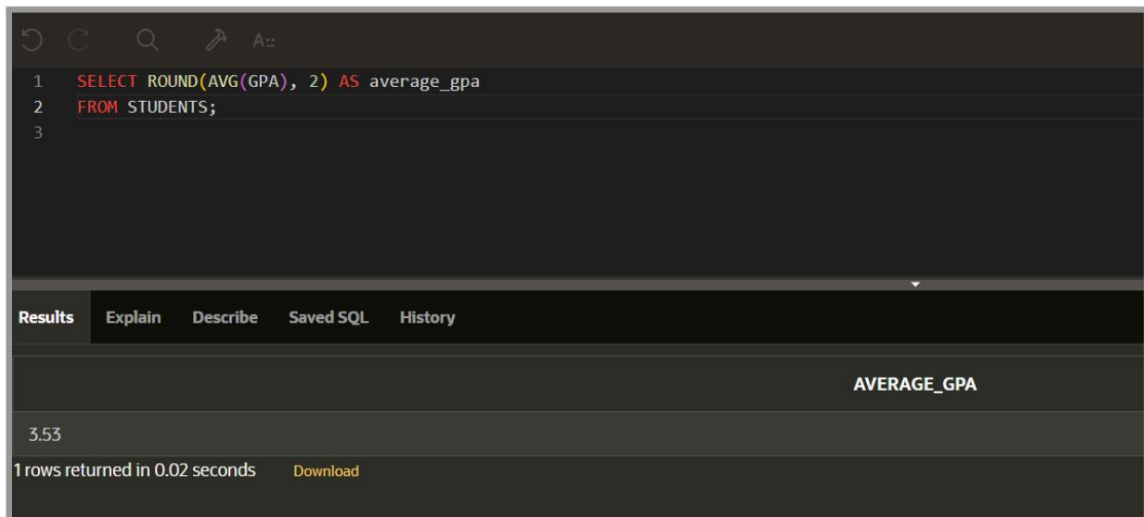
Menjumlahkan semua nilai di kolom KREDIT.

**Analogi:**

Menghitung total jumlah kredit yang ditawarkan oleh semua mata kuliah.

**2.3 Fungsi AVG() Fungsi**

AVG() menghitung nilai rata-rata dari kolom yang berupa angka.



```

1 SELECT ROUND(AVG(GPA), 2) AS average_gpa
2 FROM STUDENTS;
3

```

| AVERAGE_GPA |
|-------------|
| 3.53        |

1 rows returned in 0.02 seconds [Download](#)

**Penjelasan:**

Hitunglah rata-rata IPK seluruh siswa dan bulatkan ke dua desimal.

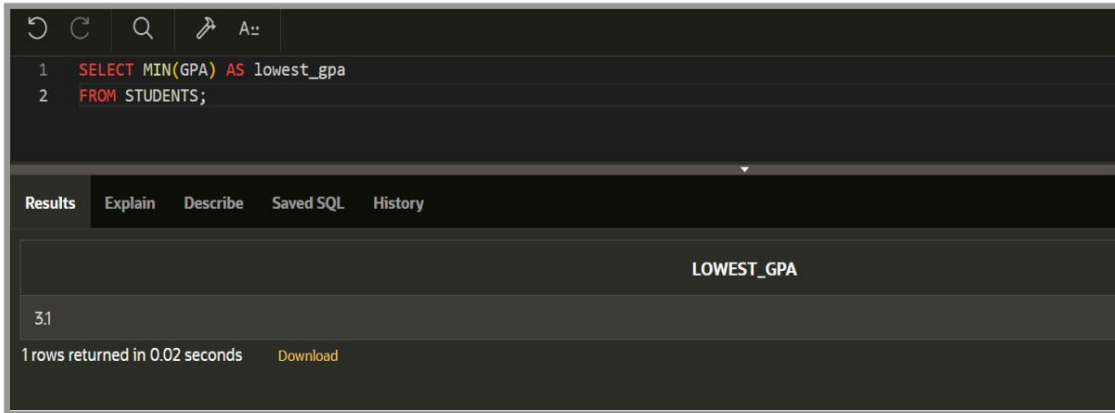


**Analogi:**

Hitunglah rata-rata IPK suatu kelas.

## 2.4 Fungsi MIN()

Fungsi MIN() mengembalikan nilai terkecil atau menemukan nilai terkecil dalam sebuah kolom.



```

1 SELECT MIN(GPA) AS lowest_gpa
2 FROM STUDENTS;

```

| LOWEST_GPA |
|------------|
| 3.1        |

1 rows returned in 0.02 seconds [Download](#)

**Penjelasan:**

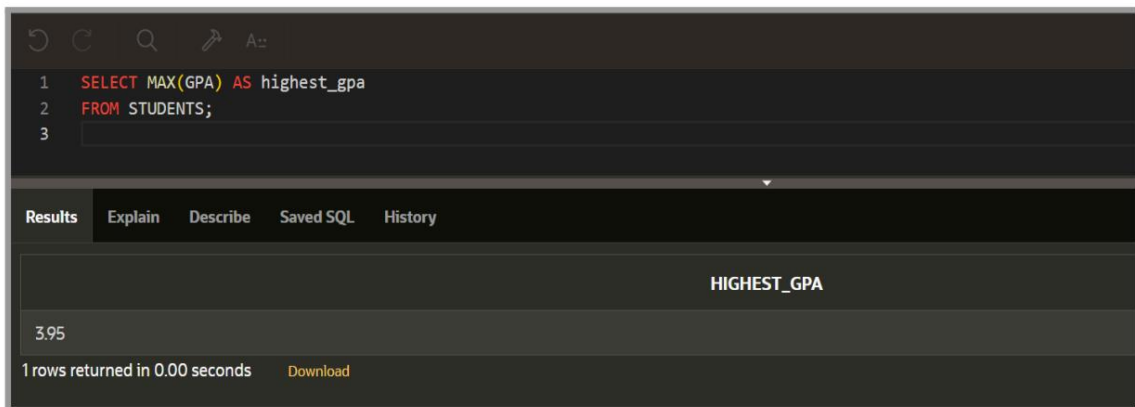
Temukan IPK terendah dalam tabel MAHASISWA.

**Analogi:**

Carilah nilai ujian terendah di kelas.

## 2.5 Fungsi MAX()

Fungsi MAX() mengembalikan nilai atau menemukan nilai terbesar dalam sebuah kolom.



```

1 SELECT MAX(GPA) AS highest_gpa
2 FROM STUDENTS;
3

```

| HIGHEST_GPA |
|-------------|
| 3.95        |

1 rows returned in 0.00 seconds [Download](#)

**Penjelasan:**

Temukan IPK tertinggi dalam tabel MAHASISWA.

**Analogi:**

Carilah nilai ujian tertinggi di kelas.



### 3. Klausula Pendukung untuk Fungsi Agregat

#### 3.1 Klausul WHERE

Klausul WHERE menyaring baris sebelum fungsi agregasi diterapkan, jadi WHERE adalah termasuk dalam kategori penyaringan data.

##### Operator Perbandingan

| Operator | Arti                              |
|----------|-----------------------------------|
| =        | Setara                            |
| >        | Lebih besar dari                  |
| <        | Kurang dari                       |
| >=       | Lebih besar dari atau sama dengan |
| <=       | Kurang dari atau sama dengan      |
| <>       | Tidak sama                        |

##### Operator Logika

| Operator | Arti                                            |
|----------|-------------------------------------------------|
| DAN      | Kedua syarat tersebut harus terpenuhi.<br>BENAR |
| ATAU     | Salah satu syaratnya adalah...<br>BENAR         |
| BUKAN    | Membalikkan suatu kondisi                       |

Salah satu contoh operator perbandingan:



```

1 SELECT COUNT(*) AS students_2024
2 FROM STUDENTS
3 WHERE ENROLLMENTYEAR = 2024;
4

```

Results Explain Describe Saved SQL History

| STUDENTS_2024 |
|---------------|
| 2             |

1 rows returned in 0.02 seconds [Download](#)

Salah satu contoh operator logika:

```

1 SELECT COUNT(*) AS active_high_gpa
2 FROM STUDENTS
3 WHERE GPA >= 3.5 AND STATUS = 'Active';
4

```

Results Explain Describe Saved SQL History

| ACTIVE_HIGH_GPA |
|-----------------|
| 3               |

1 rows returned in 0.00 seconds [Download](#)

### Analogi:

Analogi dalam contoh operator perbandingan:

- Penyaringan Mahasiswa Tahun Akademik 2024. Analogi

dalam contoh operator logika: • Hanya

menyaring mahasiswa aktif dengan IPK tinggi sebelum melakukan perhitungan.

## 3.2 Klausul GROUP BY

Klausul GROUP BY mengelompokkan baris dengan nilai yang sama sehingga fungsi agregasi dapat diterapkan per kelompok. Dengan menggunakan GROUP BY, Anda dapat menerapkan fungsi agregasi (seperti SUM, COUNT, AVG, MIN, atau MAX) ke setiap kelompok baris yang dihasilkan dengan mengurutkan berdasarkan kolom tertentu.



Language SQL Rows 10 Clear Command Find Tables Save Run

```

1 SELECT MAJORID, COUNT(*) AS students_count
2 FROM STUDENTS
3 GROUP BY MAJORID;
4

```

Results Explain Describe Saved SQL History

| MAJORID | STUDENTS_COUNT |
|---------|----------------|
| 101     | 3              |
| 103     | 1              |
| 102     | 1              |

3 rows returned in 0.00 seconds Download

**Penjelasan:**

Setiap program studi (MAJORID) membentuk sebuah kelompok, dan fungsi COUNT(\*) menghitung jumlah mahasiswa MAJORID di setiap program studi.

**Analogi:**

Mengelompokkan mahasiswa berdasarkan jurusan dan menghitung setiap kelompok secara terpisah.

**3.3 Klausul HAVING**

HAVING digunakan untuk menentukan pengelompokan yang akan ditampilkan. Selain itu, having dapat membatasi pengelompokan berdasarkan informasi Fungsi Agregat.

Results Explain Describe Saved SQL History

```

1 SELECT MAJORID, COUNT(*) AS students_count
2 FROM STUDENTS
3 GROUP BY MAJORID
4 HAVING COUNT(*) > 2;
5

```

| MAJORID | STUDENTS_COUNT |
|---------|----------------|
| 101     | 3              |

1 rows returned in 0.00 seconds Download

**Penjelasan:**

Hanya jurusan dengan lebih dari 2 mahasiswa yang ditampilkan.

**Perbedaan dari DI MANA:**

- DI MANA ÿ memfilter baris
- MEMILIKI ÿ grup filter



**Analogi:**

Setelah mengelompokkan mahasiswa berdasarkan ID jurusan mereka, hanya tampilkan jurusan yang memiliki banyak mahasiswa.

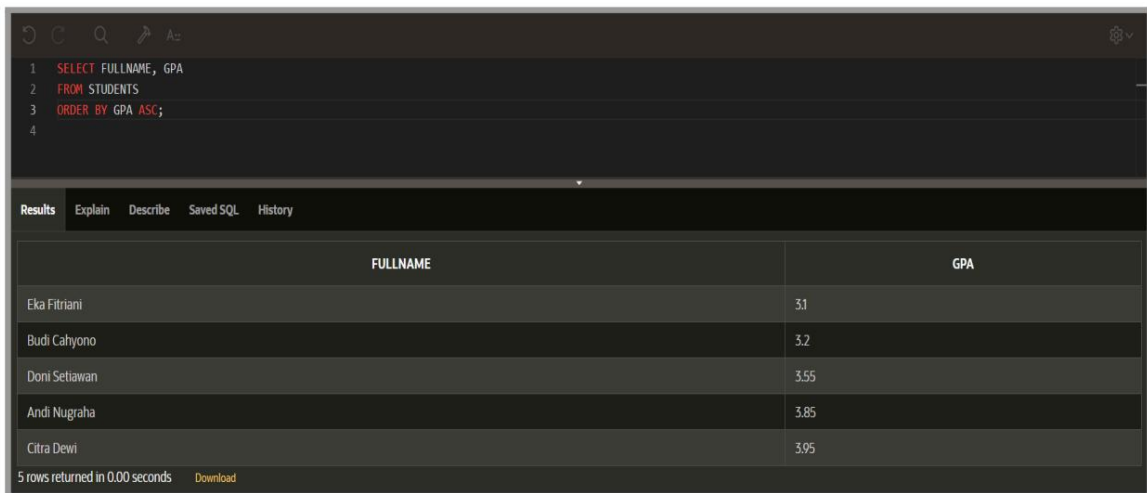
**4. Mengurutkan Data (Mengurutkan Hasil Kueri)****4.1 Konsep Pengurutan Data**

Pengurutan mengatur hasil kueri ke dalam urutan tertentu menggunakan klausa ORDER BY.

Pengurutan diterapkan **setelah** data dipilih dan diagregasi.

**4.2 Pengurutan Naik (ASC)**

Urutan menaik mengurutkan data dari yang terkecil hingga yang terbesar.



```

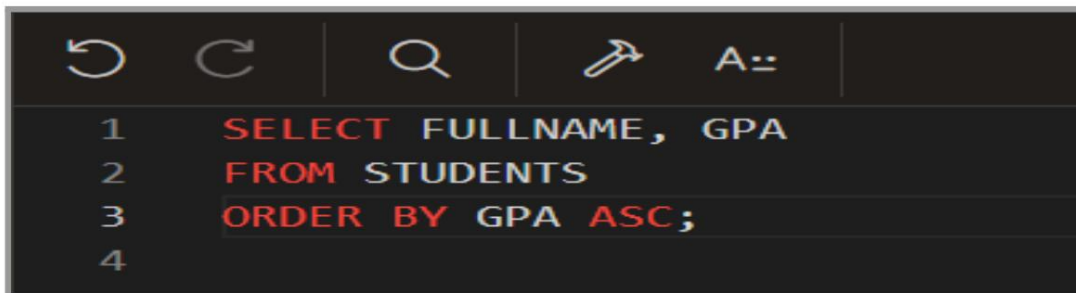
1 SELECT FULLNAME, GPA
2 FROM STUDENTS
3 ORDER BY GPA ASC;
4

```

| FULLNAME      | GPA  |
|---------------|------|
| Eka Fitriani  | 3.1  |
| Budi Cahyono  | 3.2  |
| Doni Setiawan | 3.55 |
| Andi Nugraha  | 3.85 |
| Citra Dewi    | 3.95 |

5 rows returned in 0.00 seconds [Download](#)

Sintaks Kueri jika Anda tidak melihat gambar di atas:



```

1 SELECT FULLNAME, GPA
2 FROM STUDENTS
3 ORDER BY GPA ASC;
4

```

**Penjelasan:**

Hasil diurutkan dari nilai terkecil hingga nilai terbesar. Baris dengan IPK lebih rendah muncul terlebih dahulu, diikuti oleh nilai IPK yang lebih tinggi.

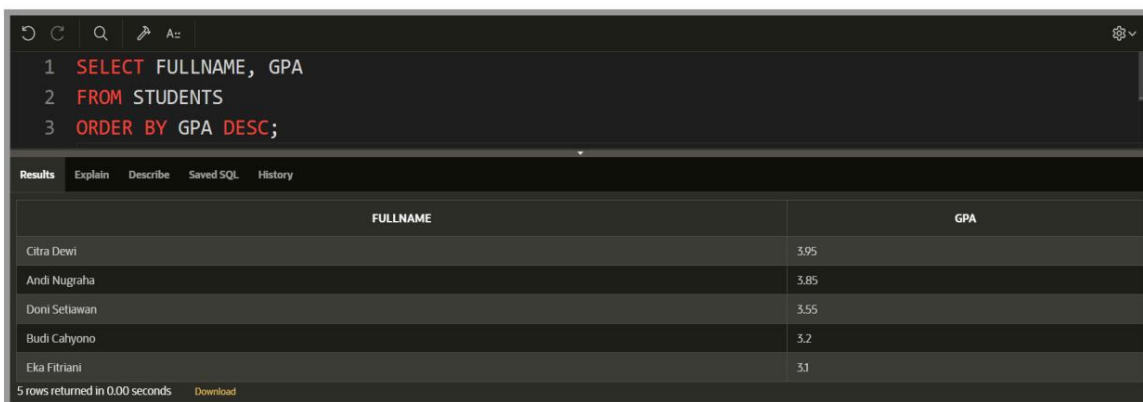


**Analogi:**

Mengurutkan IPK dari terendah ke tertinggi.

**4.3 Pengurutan Menurun (DESC)**

Pengurutan menurun adalah pengurutan data dari yang terbesar ke yang terkecil.



```

1 SELECT FULLNAME, GPA
2 FROM STUDENTS
3 ORDER BY GPA DESC;

```

| FULLNAME      | GPA  |
|---------------|------|
| Citra Dewi    | 3.95 |
| Andi Nugraha  | 3.85 |
| Doni Setiawan | 3.55 |
| Budi Cahyono  | 3.2  |
| Eka Fitriani  | 3.1  |

5 rows returned in 0.00 seconds [Download](#)

**Penjelasan:**

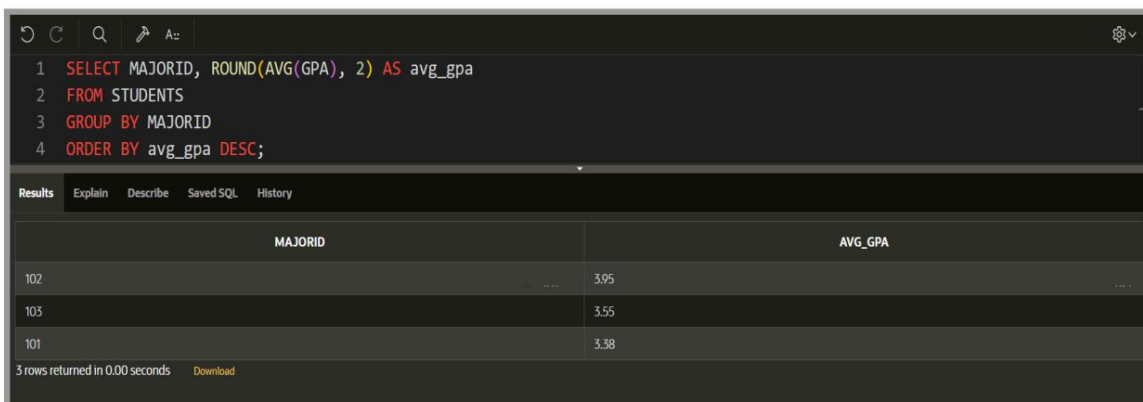
Hasilnya diurutkan **dari nilai terbesar ke nilai terkecil**. Baris dengan IPK lebih tinggi muncul terlebih dahulu, seperti daftar peringkat.

**Analogi:**

Seperti papan peringkat di mana **skor terbaik ditampilkan di bagian atas**.

**4.4 Mengurutkan Hasil Fungsi Agregat**

Pengurutan juga dapat diterapkan pada hasil agregasi.



```

1 SELECT MAJORID, ROUND(AVG(GPA), 2) AS avg_gpa
2 FROM STUDENTS
3 GROUP BY MAJORID
4 ORDER BY avg_gpa DESC;

```

| MAJORID | AVG_GPA |
|---------|---------|
| 102     | 3.95    |
| 103     | 3.55    |
| 101     | 3.38    |

3 rows returned in 0.00 seconds [Download](#)

**Penjelasan:**

Menghitung IPK rata-rata per jurusan, kemudian mengurutkan jurusan dari IPK rata-rata tertinggi ke terendah.

**Analogi:**

Pemeringkatan jurusan berdasarkan prestasi akademik.

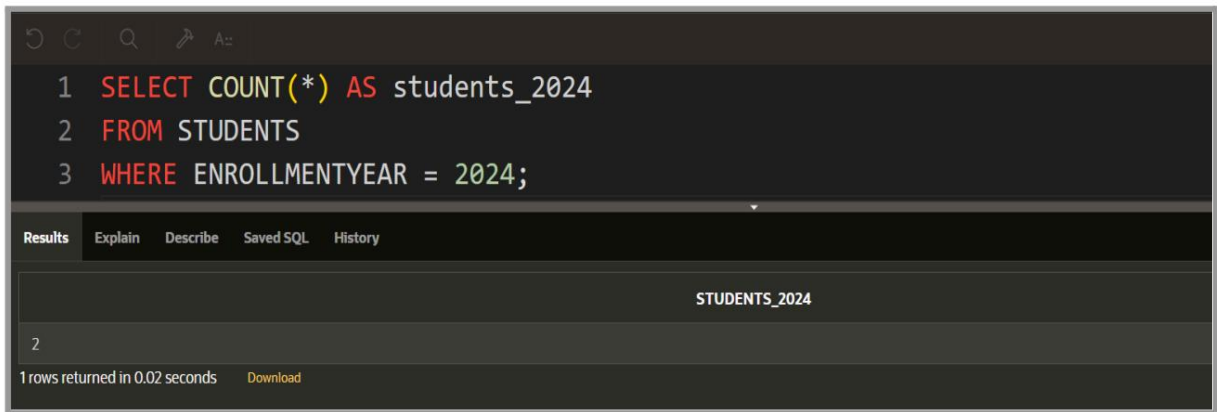


## 5. Menggabungkan Fungsi Agregat dalam Kueri SQL

Fungsi agregat jarang digunakan sendiri. Dalam analisis basis data sebenarnya, fungsi agregat dikombinasikan dengan **klausula penyaringan, pengelompokan, dan pengurutan** untuk menghasilkan hasil yang lebih bermakna. Bagian ini menjelaskan bagaimana fungsi agregat bekerja bersama dengan WHERE, GROUP BY, HAVING, dan ORDER BY.

### 5.1 Mengurutkan Hasil Fungsi Agregat

Klausul WHERE digunakan untuk **memfilter baris sebelum perhitungan agregasi dilakukan**. Hanya baris yang memenuhi syarat yang disertakan dalam perhitungan.



```

1 SELECT COUNT(*) AS students_2024
2 FROM STUDENTS
3 WHERE ENROLLMENTYEAR = 2024;

```

Results Explain Describe Saved SQL History

| STUDENTS_2024 |
|---------------|
| 2             |

1 rows returned in 0.02 seconds [Download](#)

#### Penjelasan:

- klausula WHERE ENROLLMENTYEAR = 2024 menyaring data terlebih dahulu.
- Hanya siswa yang terdaftar pada tahun 2024 yang dihitung.
- Agregasi diterapkan setelah penyaringan.

Pendekatan ini mencegah baris yang tidak relevan memengaruhi hasil.

#### Analogi:

Seperti memilih hanya siswa dari satu tahun penerimaan sebelum menghitung statistik.

### 5.2 Fungsi Agregat dengan GROUP BY

GROUP BY digunakan untuk membagi baris menjadi beberapa kelompok sehingga fungsi agregasi dihitung per kelompok, bukan untuk seluruh tabel.



**Penjelasan:** •

Kueri ini membantu kami membandingkan prestasi akademik dari kelompok mahasiswa yang berbeda.

Seperti membandingkan dan menganalisis kinerja rata-rata setiap angkatan per tahun, misalnya: angkatan 2022 vs. angkatan 2023 vs. angkatan 2024.

HAVING digunakan untuk memfilter grup setelah proses agregasi selesai. Fungsinya mirip dengan WHERE, tetapi diterapkan pada tingkat grup. Misalnya, hanya memilih jurusan yang rata-rata IPK-nya melebihi ambang batas tertentu.

**Penjelasan:** •

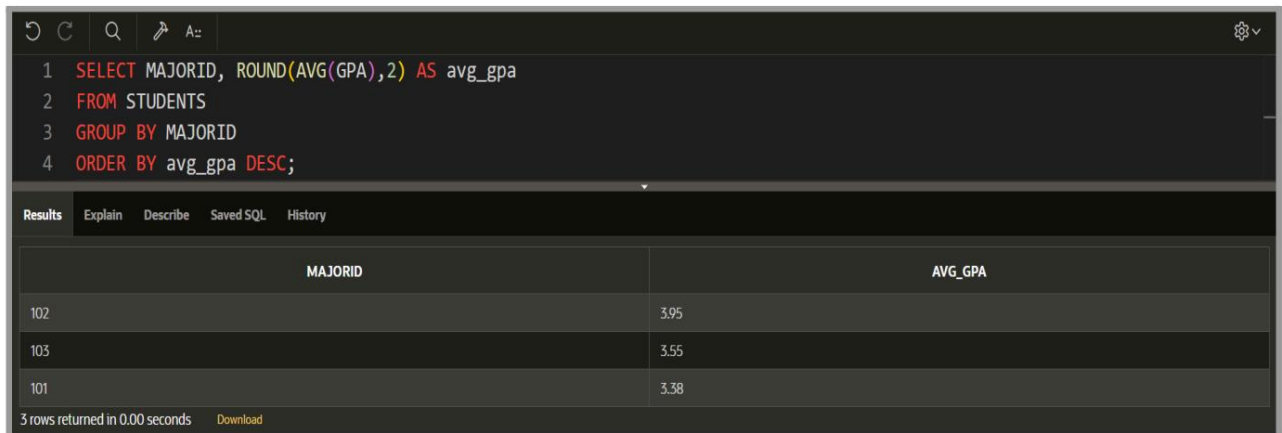
GROUP BY MAJORID mengelompokkan mahasiswa berdasarkan jurusan • AVG(GPA) menghitung rata-rata IPK per jurusan • HAVING AVG(GPA) >= 3.2 hanya mempertahankan jurusan dengan IPK tinggi. WHERE tidak dapat digunakan dalam kasus ini karena kondisinya didasarkan pada hasil agregat.

**Analogi:**

Sama seperti menganalisis setiap jurusan terlebih dahulu, langkah selanjutnya adalah hanya menampilkan jurusan dengan prestasi akademik yang baik.

## 5.4 Fungsi Agregat dengan ORDER BY

ORDER BY berguna untuk mengurutkan hasil agregasi akhir, baik dalam urutan menaik (ASC) atau menurun (DESC).



```

1 SELECT MAJORID, ROUND(AVG(GPA),2) AS avg_gpa
2 FROM STUDENTS
3 GROUP BY MAJORID
4 ORDER BY avg_gpa DESC;

```

| MAJORID | AVG_GPA |
|---------|---------|
| 102     | 3.95    |
| 103     | 3.55    |
| 101     | 3.38    |

3 rows returned in 0.00 seconds [Download](#)

**Penjelasan:**

- Kueri ini menghitung rata-rata IPK per program studi.
- ORDER BY avg\_gpa DESC digunakan untuk mengurutkan hasil dari yang tertinggi hingga terendah.
- Rata-rata IPK terendah (DESC).
- Hal ini memudahkan analisis program studi dengan kinerja terbaik.

**Analogi:**

Seperti membuat daftar peringkat di mana skor tertinggi ditampilkan terlebih dahulu, diikuti oleh skor terendah.

## 6. Fungsi Tambahan (Pendahuluan)

Fungsi agregasi tambahan digunakan untuk melakukan analisis data yang lebih spesifik. Fungsi-fungsi ini tidak selalu diperlukan dalam operasi SQL dasar, tetapi berguna dalam situasi di mana dibutuhkan peringkasan data yang lebih cepat dan lebih canggih. Fungsi-fungsi yang dibahas dalam bagian ini meliputi LISTAGG(), MEDIAN(), RANK(), dan CASE WHEN dalam operasi agregasi.

### 6.1 Fungsi LISTAGG()

Fungsi LISTAGG() digunakan untuk menggabungkan beberapa nilai teks dari baris yang berbeda menjadi satu string, biasanya dipisahkan oleh koma atau spasi. Fungsi ini berguna untuk menampilkan data teks yang dikelompokkan dalam bentuk yang ringkas.



```

1 SELECT MAJORID,
2       LISTAGG(FULLNAME, ', ' )
3       WITHIN GROUP (ORDER BY FULLNAME) AS student_list
4 FROM STUDENTS
5 GROUP BY MAJORID;

```

| MAJORID | STUDENT_LIST                             |
|---------|------------------------------------------|
| 101     | Andi Nugraha, Budi Cahyono, Eka Fitriani |
| 102     | Citra Dewi                               |
| 103     | Doni Setiawan                            |

**Penjelasan: •**

Mahasiswa dikelompokkan berdasarkan JURUSAN •

LISTAGG() menggabungkan semua nama mahasiswa dalam jurusan yang sama • Hasilnya menampilkan satu baris per jurusan yang berisi daftar nama

**Analogi:**

Seperti menulis daftar kehadiran kelas dalam satu baris, misalnya: "Anna, Budi, Clara, David".

**6.2 Fungsi MEDIAN()**

Fungsi MEDIAN() mengembalikan nilai tengah dalam kumpulan data numerik yang telah diurutkan. berguna ketika nilai rata-rata dapat dipengaruhi oleh nilai ekstrem (outlier).

```

1 SELECT MAJORID,
2       MEDIAN(GPA) AS median_gpa
3 FROM STUDENTS
4 GROUP BY MAJORID;

```

| MAJORID | MEDIAN_GPA |
|---------|------------|
| 101     | 3.2        |
| 102     | 3.95       |
| 103     | 3.55       |

**Penjelasan: •**

Mahasiswa dikelompokkan berdasarkan

jurusan • MEDIAN(GPA) menemukan **nilai GPA tengah** di setiap

kelompok • Hasilnya mewakili **tingkat kinerja rata-rata** setiap jurusan

**Analogi:**

Jika IPK diurutkan seperti

2,8, 3,0, 3,2, 3,5, 3,7,

maka **nilai tengah (3,2)** adalah median.



### 6.3 Fungsi RANK()

RANK() adalah fungsi jendela yang digunakan untuk menetapkan nomor peringkat pada baris berdasarkan suatu kriteria. urutan tertentu. Jika dua baris memiliki nilai yang sama, keduanya akan menerima peringkat yang sama.

```

1 SELECT NIM, FULLNAME, GPA,
2        RANK() OVER (ORDER BY GPA DESC) AS gpa_rank
3 FROM STUDENTS;

```

| NIM     | FULLNAME      | GPA  | GPA_RANK |
|---------|---------------|------|----------|
| 2024002 | Citra Dewi    | 3.95 | 1        |
| 2024001 | Andi Nugraha  | 3.85 | 2        |
| 2023020 | Doni Setiawan | 3.55 | 3        |
| 2023015 | Budi Cahyono  | 3.2  | 4        |
| 2022030 | Eka Fitriani  | 3.1  | 5        |

#### Penjelasan: •

Baris diurutkan berdasarkan IPK dari tertinggi ke terendah • IPK

tertinggi mendapat **peringkat 1** • Siswa

dengan IPK yang sama mendapat peringkat yang sama **Analogi:**

Seperti

tabel peringkat kompetisi di mana dua siswa dapat berbagi tempat pertama jika skor mereka sama.

### 6.4 Menggunakan CASE WHEN dalam Fungsi Agregat

CASE WHEN digunakan untuk menerapkan logika dalam kueri agregasi. Fitur ini memungkinkan perhitungan atau penjumlahan hanya dilakukan pada baris yang memenuhi kondisi tertentu.

```

1 SELECT
2     SUM(CASE WHEN GPA >= 3.5 THEN 1 ELSE 0 END) AS high_gpa_students,
3     SUM(CASE WHEN GPA < 3.5 THEN 1 ELSE 0 END) AS regular_gpa_students
4 FROM STUDENTS;

```

| HIGH_GPA_STUDENTS | REGULAR_GPA_STUDENTS |
|-------------------|----------------------|
| 3                 | 2                    |

#### Penjelasan: •

Setiap kondisi menghasilkan 1 (benar) atau 0 (salah) •

SUM() menghitung berapa banyak baris yang memenuhi kondisi •

Dua kategori berbeda dihitung dalam **satu kueri Analogi:** Seperti menghitung

“Berapa banyak

siswa yang memiliki **IPK di atas 3,5** dan berapa banyak yang memiliki **IPK di bawah 3,5**”

tanpa menjalankan dua kueri terpisah.



## TABEL RINGKASAN BAHAN

| Pertanyaan                                                                                                                     | Cara Membaca dalam Bahasa Indonesia (Analogi Singkat)                                       |
|--------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| SELECT COUNT(*) FROM STUDENTS;                                                                                                 | Pilih dan hitung semua baris data dari tabel SISWA.                                         |
| SELECT SUM(CREDITS) FROM COURSES;                                                                                              | Pilih dan jumlahkan semua nilai kolom CREDITS dari tabel COURSES.                           |
| SELECT AVG(GPA) FROM STUDENTS;                                                                                                 | Pilih dan hitung nilai rata-rata dari kolom IPK pada tabel SISWA.                           |
| SELECT MIN(GPA) FROM STUDENTS;                                                                                                 | Pilih dan ambil nilai IPK yang paling kecil dari tabel SISWA.                               |
| PILIH MAKS(IPK) DARI MAHASISWA;                                                                                                | Pilih dan ambil nilai IPK yang paling besar dari tabel SISWA.                               |
| SELECT COUNT(*) FROM STUDENTS<br>DI MANA TAHUN PENDAFTARAN = 2024;                                                             | Pilih dan hitung jumlah siswa dari tabel SISWA yang tahun masuknya sama dengan 2024.        |
| SELECT COUNT(*) FROM STUDENTS WHERE<br>GPA >= 3.5 AND STATUS = 'Active';                                                       | Pilih dan hitung siswa yang IPK-nya 3.5 atau lebih dan statusnya Aktif.                     |
| SELECT MAJORID, COUNT(*) FROM<br>KELOMPOK MAHASISWA BERDASARKAN JURUSAN;                                                       | Kelompokkan siswa berdasarkan MAJORID, lalu hitung jumlah siswa di setiap jurusan.          |
| SELECT MAJORID, AVG(GPA) FROM<br>KELOMPOK MAHASISWA BERDASARKAN JURUSAN;                                                       | Kelompokkan siswa berdasarkan MAJORID, lalu hitung rata-rata IPK di setiap jurusan.         |
| SELECT MAJORID, COUNT(*) FROM<br>KELOMPOK MAHASISWA OLEH MAJORID HAVING<br>HITUNG(*) > 2;                                      | Setelah mendaftar per MAJORID, tampilkan hanya jurusan yang jumlah siswanya lebih dari dua. |
| PILIH NAMA LENGKAP, IPK DARI MAHASISWA<br><small>Urutkan berdasarkan IPK (Indeks Prestasi Kumulatif) secara Ascending;</small> | Tampilkan nama dan IPK mahasiswa, lalu urutkan dari IPK terkecil ke terbesar.               |
| PILIH NAMA LENGKAP, IPK DARI MAHASISWA<br><small>Diurutkan berdasarkan IPK menurun;</small>                                    | Tampilkan nama dan IPK mahasiswa, lalu urutkan dari IPK terbesar ke terkecil.               |
| SELECT MAJORID, AVG(GPA) FROM                                                                                                  | Kelompokkan siswa per MAJORID,                                                              |

|                                                                                                                         |                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| KELOMPOK MAHASISWA BERDASARKAN URUTAN JURUSAN OLEH Rata-rata (IPK) Dekade;                                              | hitung rata-rata IPK tiap jurusan, lalu urutkan dari rata-rata tertinggi ke terendah.             |
| SELECT MAJORID, LISTAGG(FULLNAME, ', ' ) DALAM GRUP (URUTKAN BERDASARKAN (NAMA LENGKAP) DARI KELOMPOK SISWA OLEH MAYOR; | Kelompokkan siswa per MAJORID, lalu gabungkan semua nama siswa dalam satu baris, pisahkan koma.   |
| SELECT MAJORID, MEDIAN(GPA) FROM KELOMPOK MAHASISWA BERDASARKAN JURUSAN;                                                | Kelompokkan siswa per MAJORID, lalu ambil nilai IPK tengah dari setiap kelompok.                  |
| SELECT NIM, FULLNAME, GPA, RANK() LEBIH DARI (URUTKAN BERDASARKAN IPK MENURUN) DARI SISWA;                              | Tampilkan data mahasiswa dan beri peringkat berdasarkan IPK dari yang tertinggi ke yang terendah. |
| SELECT SUM(CASE WHEN GPA >= 3.5 THEN 1 ELSE 0 END) FROM STUDENTS;                                                       | Hitung jumlah mahasiswa yang IPK-nya lebih besar atau sama dengan 3,5.                            |



## KEGIATAN PRAKTIK

Untuk Kegiatan Praktik ini, kita akan melanjutkan skema toko online yang telah kita kerjakan sebelumnya.

**Modul 4. Mohon perhatikan dan ikuti langkah-langkah berikut:**

### Studi Kasus — Pelanggan Baru dan Transaksi Baru

Toko online tersebut telah mencatat beberapa pelanggan baru dan transaksi pembelian baru.

Anda diminta untuk memasukkan data berikut ke dalam sistem.

Siswa harus menggunakan nilai ID yang diberikan agar tugas selanjutnya dapat berjalan dengan benar.

#### 1. Langkah 1 - Memasukkan Pelanggan Baru (PELANGGAN)

Tambahkan pelanggan baru berikut:

| ID Pelanggan | Nama          | E-mail        |
|--------------|---------------|---------------|
| Tahun 2002   | Lina Mahendra | lina@mail.com |
| Tahun 2003   | Arif Nugroho  | arif@mail.com |

Tambahkan kueri ini ke Oracle 11g xe:

```
INSERT INTO CUSTOMERS (CustomerID, CustomerName, Email)
VALUES (2002, 'Lina Mahendra', 'lina@mail.com');

INSERT INTO CUSTOMERS (CustomerID, CustomerName, Email)
VALUES (2003, 'Arif Nugroho', 'arif@mail.com');
```

#### 2. Langkah 2 - Masukkan Pesanan Baru (ORDERS)

Tambahkan dua pesanan baru yang dibuat oleh pelanggan baru:

| ID Pesanan | ID Pelanggan | Jumlah Total |
|------------|--------------|--------------|
| 11         | Tahun 2002   | 700.000      |
| 12         | Tahun 2003   | 850.000      |



```
INSERT INTO ORDERS (OrderID, CustomerID, TotalAmount)
VALUES (11, 2002, 700000);
```

```
INSERT INTO ORDERS (OrderID, CustomerID, TotalAmount)
VALUES (12, 2003, 850000);
```

### 3. Langkah 3 - Masukkan Detail Pesanan (ORDER\_DETAILS)

Tambahkan detail pembelian produk untuk setiap pesanan:

| ID Detail Pesanan | ID Pesanan | ID Produk | Jumlah | Subtotal |
|-------------------|------------|-----------|--------|----------|
| 3                 | 11         | A001      | 1      | 150.000  |
| 4                 | 11         | A002      | 1      | 550.000  |
| 5                 | 12         | A001      | 2      | 300.000  |
| 6                 | 12         | A002      | 1      | 550.000  |

```
INSERT INTO ORDER_DETAILS (OrderDetailID, OrderID, ProductID, Quantity, Subtotal)
VALUES (3, 11, 'A001', 1, 150000);
```

```
INSERT INTO ORDER_DETAILS (OrderDetailID, OrderID, ProductID, Quantity, Subtotal)
VALUES (4, 11, 'A002', 1, 550000);
```

```
INSERT INTO ORDER_DETAILS (OrderDetailID, OrderID, ProductID, Quantity, Subtotal)
VALUES (5, 12, 'A001', 2, 300000);
```

```
INSERT INTO ORDER_DETAILS (OrderDetailID, OrderID, ProductID, Quantity, Subtotal)
VALUES (6, 12, 'A002', 1, 550000);
```

Setelah menyelesaikan langkah ini, kita melanjutkan ke tahap kueri dermaga menggunakan agregat. fungsi.



## SUMBER BELAJAR DAN TUTORIAL

---

[LINK\\_W3SCHOOLS\\_SQL](#) (Rekomendasi)

[LINK\\_DOKUMEN\\_ORACLE](#) (Rekomendasi)

[LINK\\_PELAJARIWEB](#)



## DOCK PERTANYAAN

---

Kueri dock ini merupakan kelanjutan dari tahap aktivitas praktik dan menggunakan skema yang sama. Pada bagian ini, siswa tidak diberikan kerangka kueri. Mereka hanya membaca persyaratan kasus/melihat format tabel keluaran, **kemudian menyusun kueri SQL sendiri.**

### 1. Pertanyaan 1 - Studi Kasus (Berbasis Cerita)

Studi Kasus - Laporan Pengeluaran Pelanggan

Seorang manajer toko ingin menganalisis perilaku pelanggan. Dia ingin mengetahui **berapa total uang yang telah dihabiskan setiap pelanggan** berdasarkan semua pesanan yang tercatat dalam sistem.

#### Tugas:

- Buat kueri untuk menampilkan CustomerID dan Total **Pengeluaran** untuk setiap pelanggan.
- Konsep

**Petunjuk:** • Gunakan fungsi agregasi dan kelompokkan data berdasarkan pelanggan.

### 2. Pertanyaan 2 - Studi Kasus (Berbasis Cerita)

Studi Kasus — Kinerja Penjualan Produk

Tim keuangan ingin melihat **produk mana yang memberikan kontribusi pendapatan tertinggi**. Mereka ingin menghitung **total pendapatan untuk setiap produk** berdasarkan semua transaksi pesanan.

#### Tugas:

- Buat kueri untuk menampilkan **ProductID** dan **Total Pendapatan per produk**, lalu urutkan hasil dari **yang terbesar hingga terkecil**.

**Petunjuk:**

- Gunakan ringkasan nilai subtotal dan urutkan hasil analisis.

### 3. Pertanyaan 3 — Interpretasi Tabel Output

#### Petunjuk:

Perhatikan tabel keluaran berikut:



| ID Produk | total_terjual |
|-----------|---------------|
| A002      | 3             |
| A001      | 4             |

**Tugas:**

- Buat kueri yang dapat menghasilkan output tabel seperti yang ditunjukkan di atas.

**Pola pikir yang sedang diuji:**

- memahami **PENGELOMPOKAN BERDASARKAN**
- Memahami **SUM / COUNT** dalam detail transaksi
- kemampuan untuk menerjemahkan **format tabel** ke kueri

**4. Pertanyaan 4 — Interpretasi Tabel Output****Petunjuk:**

Perhatikan tabel keluaran berikut:

| ID Pelanggan | jumlah pesanan |
|--------------|----------------|
| 1001         | 1              |
| Tahun 2003   | 1              |
| Tahun 2002   | 1              |

**Tugas:**

- Buat kueri yang dapat menghasilkan output tabel seperti yang ditunjukkan di atas.

**Konsep yang diarahkan secara implisit:**

- **JUMLAH** per pelanggan
- **KELOMPOKAN BERDASARKAN** pelanggan



## PENUGASAN

### TUGAS 1 Untuk

tugas pertama ini, gunakan skema sesuai dengan tema yang Anda pilih di modul sebelumnya (skema yang telah diisi dalam file spreadsheet).

#### Ikuti langkah-langkah

berikut: **1. Langkah 1 — Tambahkan Data Tambahan Minimal (Dukungan Data untuk Agregat)**

##### Instruksi:

- Tambahkan setidaknya 4 entri data baru ke tabel di setiap skema (sesuai tema: supermarket, kantor, kampus, toko, dll.).

##### Persyaratan data: •

bervariasi (tidak semua nilai sama), dan •

memungkinkan penggunaan fungsi agregasi, misalnya:

- Terdapat nilai numerik yang akan dijumlahkan/dirata-ratakan, •
- Terdapat kolom kategori yang akan dikelompokkan (GROUP BY), •
- Terdapat nilai berbeda yang dapat dibandingkan (MIN/MAX).

**Tidak perlu mengikuti struktur tertentu — silakan beradaptasi dengan konteks skema tersebut.**

Tujuan dari langkah ini hanyalah untuk memastikan bahwa terdapat data yang cukup untuk analisis.

### 2. Langkah 2 — Melakukan Agregasi & Menghasilkan

**Wawasan** Dari data yang ditambahkan pada Langkah 1,

lakukan hal berikut: • **Setidaknya 3 fungsi agregasi (pilih 3 proses agregasi yang telah dipelajari dalam modul ini), misalnya:** • COUNT() •

menghitung jumlah data • GROUP BY • ringkasan

berdasarkan kategori • ORDER BY • analisis

urutan / pemeringkatan **Mahasiswa bebas menentukan**

konteks analisis sesuai dengan tema. Setelah melakukan ketiga fungsi agregasi tersebut, silakan sampaikan temuan Anda kepada asisten mengenai wawasan/interpretasi, misalnya: “Dengan menggunakan pengurutan menurun, kita dapat dengan mudah melihat produk mana yang memberikan kontribusi penjualan terbanyak.”



## TUGAS 2 (DEMO LANGSUNG)

- Siswa akan diminta untuk membuat tabel sederhana sesuai dengan instruksi asisten.  
selama praktik kerja lapangan.
- Setelah tabel berhasil dibuat, mahasiswa diminta untuk memasukkan beberapa data (sejumlah kecil data, tidak berlebihan) sesuai dengan contoh yang diberikan oleh asisten.
- Waktu maksimal yang diberikan kepada setiap siswa untuk menyelesaikan tugas adalah 20 menit.  
setelah instruksi diberikan.
- Jika siswa gagal mengikuti instruksi, asisten akan memberikan beberapa pertanyaan terkait penggunaan **Fungsi Agregat (COUNT, SUM, AVG, GROUP BY, HAVING, ORDER BY, dll.)**.

### Catatan Penting: •

- Mahasiswa diwajibkan mempelajari materi modul terlebih dahulu sebelum praktikum.  
dimulai.
- Membuka catatan, buku, perangkat lain, atau materi referensi apa pun selama demo langsung tidak diperbolehkan.  
Tidak diizinkan.
- **Bagi siswa yang tidak dapat menyelesaikan semua instruksi demo, pertanyaan tambahan akan diberikan untuk meningkatkan skor mereka. Terdapat empat pertanyaan yang mencakup materi dasar agregasi dan pengurutan data. Harap pelajari materi ini juga!**



## RUBRIK PENILAIAN PRAKTIKUM

| Aspek Penilaian                | Poin             |
|--------------------------------|------------------|
| <b>Dock Kueri</b>              | <b>Total 10%</b> |
| <b>Tugas 1</b>                 | <b>Total 20%</b> |
| Tambahkan Data                 | 5%               |
| Lakukan Kueri Agregat          | 5%               |
| Jelaskan tiga agregat          | 10%              |
| <b>Tugas 2 (Demo Langsung)</b> | <b>Total 70%</b> |
| Buat Tabel + Tambahkan Data    | 5%               |
| Agregat Dasar                  | 10%              |
| Mengurutkan Data               | 15%              |
| Menggabungkan Agregat          | 20%              |
| Fungsi Tambahan                | 20%              |

### Catatan Penting tentang Pelaksanaan Praktikum

Selama pekan **Query Dock**, mahasiswa didorong untuk **mengajukan pertanyaan** tentang modul 5. materi yang belum mereka pahami. Sesi ini tidak hanya menguji sintaksis, tetapi juga membantu Para siswa memperdalam pemahaman mereka tentang konsep SQL melalui diskusi dengan para asisten.

Skor Query **Dock memiliki persentase kecil**, sehingga fokus utama penilaian tersebut adalah... Keberhasilan tetap bergantung pada **pemahaman materi** dan kemampuan siswa untuk menjelaskan logika dari pertanyaan tersebut.

Untuk setiap item penilaian pada rubrik:

- 100 ÿ Jawaban yang benar dan percaya diri / penjelasan yang jelas
- 50 ÿ Jawaban sebagian benar / masih ragu
- 0 ÿ Tidak dapat menjawab / tidak mengikuti instruksi

Dengan skema ini, penilaian dilakukan secara objektif dan konsisten sesuai dengan levelnya. dari penguasaan siswa.

